BiFed-S Voice Auth Demo — Project Status Report

Generated: 2026-11-02 · 16:25 IST

🗺️ Overall Completion

```
Phase 1 — Project Inspection & Planning  ████████████████████  100% ✅
Phase 2 — Backend Foundation             ████████████████████  100% ✅
Phase 3 — Frontend Application           ████████████████████  100% ✅
Phase 4 — ML Model Integration           ░░░░░░░░░░░░░░░░░░░░    0% ⏳
Phase 5 — End-to-End Testing & Polish    ░░░░░░░░░░░░░░░░░░░░    0% ⏳
```

Summary: Phases 1–3 complete. Full-stack application is live and functional. ML inference is modularly stubbed — awaiting your model weights and scripts.

Phase 1 — Project Inspection & Planning ✅ COMPLETE

- Inspected existing `BiFed-S-Web/` directory (was empty `backend/` and `frontend/` folders)
- Confirmed no existing code, no venv, no model files, no node.js
- Designed complete architecture: single-page React frontend + FastAPI backend
- Defined strict 6-stage pipeline order: `Standardization → AASIST → DeepFilterNet3 → Silero VAD → ECAPA-TDNN → Decision`
- Identified all missing dependencies

Phase 2 — Backend Foundation ✅ COMPLETE

Server & Configuration

File	Purpose	Status
`backend/app/main.py`	FastAPI app, CORS, lifespan startup	✅ Running
`backend/app/config.py`	All paths, thresholds via Pydantic Settings	✅
`backend/app/database.py`	SQLite init, CRUD for enrollments + experiments	✅
`backend/requirements.txt`	Core deps installed; ML deps commented out	✅

API Endpoints

Endpoint	Method	Purpose	Status
`/api/health`	GET	Backend liveness, version	✅ Functional
`/api/models/status`	GET	Real-time model readiness per stage	✅ Functional
`/api/enrollment`	GET	List all enrolled speakers	✅ Functional
`/api/enrollment`	POST	Enroll new speaker (audio + speaker_id)	✅ Functional
`/api/enrollment/{id}`	DELETE	Remove a speaker's reference embedding	✅ Functional
`/api/authenticate`	POST	Run full pipeline on submitted audio	✅ Functional
`/api/history`	GET	Paginated experiment history from SQLite	✅ Functional
`/api/history/stats`	GET	Aggregate stats (attempts, outcomes)	✅ Functional
`/api/history`	DELETE	Clear all experiment history	✅ Functional

Services

File	Purpose	Status
`services/audio_standardization.py`	Stage 1: decode audio, mono, 16 kHz WAV	✅ **Fully Functional**
`services/pipeline_service.py`	Orchestrates all 6 stages sequentially	✅ Wired (stubs for stages 2–6)
`services/enrollment_service.py`	Overwrite confirmation, embedding storage	✅ Functional
`services/authentication_service.py`	Cosine similarity, decision, audit log	✅ Functional (needs real embeddings)

ML Model Stubs (Modular — Awaiting Weights)

File	Model	What's Built	What's Missing
`models/aasist.py`	AASIST Anti-Spoofing	Full stub, ONNX load path wired	`models_weights/aasist/AASIST.onnx`
`models/deepfilternet.py`	DeepFilterNet3 Enhancement	Full stub, 48kHz up/down resample logic	`deepfilternet` pip package
`models/silero_vad.py`	Silero VAD	Full stub, ONNX load path wired	`models_weights/silero_vad/silero_vad.onnx`
`models/ecapa.py`	ECAPA-TDNN Embedding	Full stub, SpeechBrain load path wired	`models_weights/ecapa_tdnn/` checkpoint
`models/base.py`	Abstract base class	✅ Complete	—

Storage & Database

- `storage/bifed_experiments.db` — SQLite database created ✅
- `storage/references/` — Folder for speaker embedding `.npy` files ✅
- `storage/temp/` — Temp folder for in-flight audio processing ✅
- `models_weights/` — Empty, awaiting your model files ⏳

Installed Python Packages

```
fastapi  uvicorn  pydantic  numpy  pandas
python-multipart  python-dotenv  watchfiles  websockets
```

> ML packages NOT yet installed: `onnxruntime`, `torch`, `torchaudio`, `speechbrain`, `scipy`, `soundfile`, `deepfilternet`, `imageio-ffmpeg`

Phase 3 — Frontend Application ✅ COMPLETE

Infrastructure

File	Purpose	Status
`frontend/package.json`	React 18, Vite 5, TS, Tailwind, Recharts, Lucide	✅
`frontend/vite.config.ts`	Port 5173, `/api` proxy → backend 8000	✅ Running
`frontend/tailwind.config.js`	Dark navy theme, custom colors	✅
`frontend/src/services/api.ts`	Typed API client for all endpoints	✅
`frontend/src/types/api.ts`	Full TypeScript types for all API contracts	✅
`frontend/src/hooks/useAudioRecorder.ts`	MediaRecorder + AudioContext level meter	✅

Dashboard Sections (All on one page)

Section	Component	Features	Status
Header	`Header.tsx`	Backend ping, model count, nav	✅
System Status	`SystemStatus.tsx`	Model cards, threshold slider	✅
Audio Input	`AudioInput.tsx`	Record + Upload tabs	✅
Speaker Enrollment	`EnrollmentPanel.tsx`	Enroll, list, delete, overwrite confirm	✅
Voice Authentication	`AuthenticationPanel.tsx`	Speaker selector, authenticate, outcomes	✅
Pipeline Visualizer	`PipelineVisualizer.tsx`	6-stage flow with live status badges	✅
Model Results	`ModelResultsPanel.tsx`	Per-model telemetry cards	✅
Decision Card	`DecisionCard.tsx`	Color-coded AUTHENTICATED / REJECTED etc.	✅
Analytics	`AnalyticsPanel.tsx`	KPI counters, PieChart, BarChart (Recharts)	✅
Experiment History	`ExperimentHistory.tsx`	Table, search, modal detail, CSV export	✅
Privacy Notice	`PrivacyNotice.tsx`	Biometric data disclosure footer	✅

Audio Recording Features

- ✅ Start / Pause / Resume / Stop recording via `MediaRecorder` API
- ✅ Live audio-level meter via `AudioContext` + `AnalyserNode`
- ✅ Recording duration timer
- ✅ Playback preview of recorded audio
- ✅ Discard and re-record option
- ✅ Microphone permission request and denial handling
- ✅ Error handling for missing mic, empty recordings, too-short clips

Audio Upload Features

- ✅ Drag-and-drop or file browser
- ✅ WAV and other browser-decodable formats
- ✅ File name, size, duration display

- ✅ Playback preview
- ✅ Remove and replace option

Frontend Build Verified

```
✓ 2376 modules transformed
✓ dist/assets/index.js  633 kB
✓ Built in 23s — Exit Code 0
```

Currently Running Services

Service	Task	URL	Status
FastAPI backend	`task-492`	http://127.0.0.1:8000	🟢 Running
Vite frontend	`task-497`	http://localhost:5173	🟢 Running

> **Open browser** → **http://localhost:5173** to use the dashboard.

Phase 4 — ML Model Integration ⏳ PENDING

> **Blocked on: your model files and Colab scripts.**

This phase requires you to provide:

What You Need to Supply

```
backend/models_weights/
├── aasist/
│   └── AASIST.onnx              ← Export from your Colab notebook
├── silero_vad/
│   └── silero_vad.onnx          ← Download from Silero GitHub releases
└── ecapa_tdnn/
    ├── hyperparams.yaml          ← SpeechBrain checkpoint files
    ├── embedding_model.ckpt
    └── classifier.ckpt
```

For DeepFilterNet3: install via `pip install deepfilternet` (no manual weight file needed).

What Will Be Done (When You Provide Files)

1. **Wire AASIST** (`models/aasist.py`) — implement `predict()` using your Colab preprocessing + ONNX Runtime inference
2. **Wire DeepFilterNet3** (`models/deepfilternet.py`) — implement `enhance()` (16kHz → 48kHz → DeepFilterNet3 → 16kHz)
3. **Wire Silero VAD** (`models/silero_vad.py`) — implement `process()` for speech activity detection + segment stats
4. **Wire ECAPA-TDNN** (`models/ecapa.py`) — implement `extract_embedding()` for 192-dim speaker vectors
5. **Fix WebM/Opus decoding** — install `imageio-ffmpeg` so browser recordings (WebM) are properly decoded
6. **Install ML dependencies** via pip: `onnxruntime torch torchaudio speechbrain scipy soundfile imageio-ffmpeg`
7. **Real cosine similarity** — authentication service already calculates it; it needs real embeddings to be meaningful

ML Packages to Install (Pending)

```
bash
pip install onnxruntime torch torchaudio speechbrain scipy soundfile deepfilternet imageio-ffmpeg
# Note: may need numpy<2.0.0 for torch/speechbrain compatibility
```

Phase 5 — Testing & Polish ⏳ PENDING

Depends on Phase 4 completion.

- End-to-end enrollment with real ECAPA embedding
- End-to-end authentication with real cosine similarity
- AASIST spoof detection on real bona fide vs. spoof samples
- DeepFilterNet3 enhancement quality check
- Silero VAD segment accuracy check
- Pipeline timing benchmarks
- Threshold calibration note (currently 0.65 — Provisional)
- Temp file cleanup verification (no raw audio retained after processing)
- Mobile responsiveness check
- README final review

Known Limitations / Issues

Issue	Severity	Fix Available
Browser MediaRecorder sends WebM/Opus (not WAV) — backend can't decode it without FFmpeg	🔴 High	Install `imageio-ffmpeg` (Phase 4)
All ML stages return `AWAITING_MODEL_INTEGRATION` — no real inference yet	🔴 High	Phase 4
Authentication threshold 0.65 is provisional — not calibrated	🟡 Medium	Phase 5 after real embeddings
`storage/temp/` has leftover audio files (not auto-cleaned from previous tests)	🟡 Low	Add cleanup job
No system Node.js — must use portable `.tools/node-v20.18.0-win-x64/`	🟡 Low	Install Node.js system-wide optionally
No system FFmpeg	🟡 Low	`imageio-ffmpeg` pip package handles it

File Tree Summary

```
BiFed-S-Web/
├── backend/
│   ├── app/
│   │   ├── main.py                    ✅
│   │   ├── config.py                  ✅
│   │   ├── database.py                ✅
│   │   ├── api/
│   │   │   ├── health.py              ✅
│   │   │   ├── enrollment.py          ✅
│   │   │   ├── authentication.py      ✅
│   │   │   └── history.py             ✅
│   │   ├── models/
│   │   │   ├── base.py                ✅
│   │   │   ├── aasist.py              ⏳ stub
│   │   │   ├── deepfilternet.py       ⏳ stub
│   │   │   ├── silero_vad.py          ⏳ stub
│   │   │   └── ecapa.py               ⏳ stub
│   │   ├── schemas/
│   │   │   ├── pipeline.py            ✅
│   │   │   ├── enrollment.py          ✅
│   │   │   ├── authentication.py      ✅
│   │   │   └── history.py             ✅
│   │   └── services/
│   │       ├── audio_standardization.py  ✅ FULLY FUNCTIONAL
│   │       ├── pipeline_service.py       ✅
│   │       ├── enrollment_service.py     ✅
│   │       └── authentication_service.py ✅
│   ├── models_weights/                ⏳ empty — awaiting your files
│   ├── storage/
│   │   ├── bifed_experiments.db       ✅ live SQLite DB
│   │   ├── references/                ✅ (empty — no speakers enrolled yet)
│   │   └── temp/                      ✅ (temp audio files accumulating)
│   ├── venv/                          ✅ Python 3.11.9
│   └── requirements.txt               ✅
├── frontend/
│   ├── src/
│   │   ├── App.tsx                    ✅
│   │   ├── components/ (11 files)     ✅
│   │   ├── hooks/useAudioRecorder.ts  ✅
│   │   ├── services/api.ts            ✅
│   │   └── types/ (2 files)           ✅
│   ├── node_modules/                  ✅ installed
│   ├── package.json                   ✅
```

```
│   └── vite.config.ts                ✅
├── README.md                         ✅
├── start_demo.ps1                    ✅
└── .tools/node-v20.18.0-win-x64/     ✅ portable Node.js
```

Next Action Required From You

> **To unlock Phase 4, provide your Colab model files.**

Steps:

1. Export `AASIST.onnx` from your Colab AASIST notebook
2. Copy your ECAPA-TDNN SpeechBrain checkpoint to `backend/models_weights/ecapa_tdnn/`
3. Download `silero_vad.onnx` from the Silero GitHub releases page
4. Share your custom preprocessing scripts if any (for AASIST or ECAPA input format)

Once you provide the files, I'll wire all 4 models and run a full end-to-end test.